

Lite Arm SDK: Repository Structure and API

- Language: C++20

Repository Structure

```
lite_arm_sdk/  
|-- CMakeLists.txt  
|-- include  
|   |-- spark.h          (main SDK class API)  
|   |-- configurator.h   (configuration API)  
|   |-- kinematics.h     (kinematics API)  
|   |-- types.h          (shared data types/enums)  
|-- examples/  
|   |-- move_single_point.cpp  
|   |-- move_path.cpp  
|   |-- set_configuration.cpp  
|   |-- teach_mode.cpp  
|   |-- playback_mode.cpp  
|   |-- calculate_kinematics.cpp
```

Core SDK API (Spark)

API	Description	Group
<code>Spark(config_prefix_path="")</code>	Create SDK client; optional path prefix for configuration files.	constructor
<code>start(), stop()</code>	Start or stop arm runtime session.	connection
<code>enableArmJoint(arm_state)</code>	Enable arm joints by 6-element boolean state array.	connection
<code>setArmSmoothingMethod(method)</code>	Select smoothing strategy for single-target arm motion.	motion mode
<code>movePos(arm_pos, v=50, t=0)</code>	Command joint-space target position.	manual motion
<code>movePosPath(path, vPath, tPath)</code>	Command sequence of joint-space waypoints.	manual motion
<code>moveVel(arm_vel, a=10, t=0)</code>	Command joint-space velocity.	manual motion
<code>moveEEPoint(pose, v=50, t=0)</code>	Move end-effector to cartesian pose (point mode).	manual motion

API	Description	Group
<code>moveEEPointPath(path, vPath, tPath)</code>	Move through end-effector pose waypoints (point mode).	manual motion
<code>moveEELine(pose, v=50, t=0)</code>	Move end-effector to cartesian pose (line mode).	manual motion
<code>moveEELinePath(path, vPath, tPath)</code>	Move through end-effector pose waypoints (line mode).	manual motion
<code>moveToolPoint(pose, v=50, t=0)</code>	Move tool frame to cartesian pose (point mode).	manual motion
<code>moveToolPointPath(path, vPath, tPath)</code>	Move tool frame through cartesian waypoints (point mode).	manual motion
<code>moveToolLine(pose, v=50, t=0)</code>	Move tool frame to cartesian pose (line mode).	manual motion
<code>moveToolLinePath(path, vPath, tPath)</code>	Move tool frame through cartesian waypoints (line mode).	manual motion
<code>goHome(v=50, t=0)</code>	Move robot arm to predefined home pose.	manual motion
<code>moveGripperPos(pos, v=50, t=0)</code>	Command gripper joint position.	manual motion
<code>startTeach(), stopTeach()</code>	Enter or exit teach mode.	teach mode
<code>startPlayback(), stopPlayback(), resetPlayback()</code>	Control playback mode lifecycle.	playback mode
<code>loadPlayback(filename)</code>	Load playback trajectory or script file.	playback mode
<code>getPos(), getVel(), getTor()</code>	Read current joint position, velocity, and torque.	feedback
<code>getEEPose(), getToolPose()</code>	Read current end-effector/tool cartesian pose.	feedback
<code>isLatestTargetReceived()</code>	Whether the latest motion target has been received.	feedback
<code>isArmJointEnabled()</code>	Read per-joint arm enable state.	status
<code>getStatus(), printStatus()</code>	Read or print aggregated runtime status (<code>SystemStatus</code>).	status
<code>getConfigurator(), getKinematics()</code>	Access specialized subsystems for config/kinematics.	subsystems

Configuration API (**Configurator**)

API	Description
<code>setToolOffset(offset), getToolOffset()</code>	Set or get tool frame offset from end-effector frame.
<code>setArmPosLimits(limit), setArmVelLimits(limit)</code>	Set joint position/velocity safety limits.
<code>getArmPosLimits(), getArmVelLimits()</code>	Get active joint safety limits.
<code>getArmMaxPos(), getArmMaxVel()</code>	Get hardware max position/velocity constraints.
<code>getArmMaxPosJump(), getArmMaxVelJump()</code>	Get anti-jump protection limits.
<code>getArmMaxPosFollowError(), getArmMaxVelFollowError(), getArmMaxTorFollowError()</code>	Get following-error thresholds.
<code>setGripperPosLimits(limit), getGripperPosLimits()</code>	Set or get gripper position limits.
<code>getGripperMaxPos()</code>	Get gripper hardware max range.

Kinematics API (**Kinematics**)

API	Description
<code>forwardKinematics(arm_pos)</code>	Compute cartesian pose from 6-joint arm state.
<code>inverseKinematics(ee_pose, arm_pos)</code>	Solve 6-joint arm state for a target cartesian pose; returns bool .
<code>eulerToQuaternion(euler)</code>	Convert intrinsic XYZ Euler angles to quaternion.
<code>quaternionToEuler(quat)</code>	Convert quaternion to intrinsic XYZ Euler angles.
<code>eeToToolPose(ee_pose, tool_offset)</code>	Transform end-effector pose into tool pose.
<code>toolToEEPose(tool_pose, tool_offset)</code>	Transform tool pose into end-effector pose.

Types (**types.h**)

Type / Enum	Description
Position , Quaternion , EulerAngle	Cartesian position, orientation, and intrinsic XYZ Euler angles.
Pose	Cartesian pose = Position + Quaternion orientation.

Type / Enum	Description
<code>JointState6f</code> / <code>JointState1f</code>	6-DOF arm joint values / 1-DOF gripper value.
<code>JointState6b</code> / <code>JointState1b</code>	6-DOF arm enable flags / gripper enable flag.
<code>JointState6u</code> / <code>JointState1u</code>	Per-joint diagnostic bitmasks for arm and gripper.
<code>Path<T></code>	Waypoint sequence (<code>std::vector<T></code>).
<code>RobotJointStatef</code> / <code>RobotJointStateb</code>	Combined arm+gripper state in float or bool forms.
<code>JointLimits<T></code> , <code>JointLimits6f</code> / <code>JointLimits1f</code>	Per-joint min/max limits for arm and gripper.
<code>SmoothingMethod</code>	<code>kLinear</code> , <code>kCos</code> , <code>kCubic</code> , <code>kQuintic</code> , <code>kNone</code> .
<code>RobotState</code>	<code>kStartup</code> , <code>kIdle</code> , <code>kMoving</code> , <code>kSettling</code> , <code>kError</code> , <code>kRecovery</code> , <code>kShutdown</code> .
<code>PlanResult</code>	<code>kSuccess</code> , <code>kPoseNotReachable</code> , <code>kLinearPathFailed</code> .
<code>SystemStatus</code>	Aggregated status: <code>robot_state</code> , <code>plan_result</code> , <code>robot_diagnostic_flags</code> , per-joint diagnostic flags.
<code>DiagnosticFlags</code>	Bitmask flags for target/actuator saturation, jump limits, boundary safety, planner events, and hardware/tracking faults.